Exercício 1

Tema: “Checkout de E-commerce sem `switch` gigante”

Enunciado

Você recebeu o seguinte código em um Console App:

Código inicial

```
using System;

public enum TipoPagamento
{
    Debito,
    Credito,
    Pix
}

public class CarrinhoDeCompras
{
    public decimal ValorTotal { get; private set; }

    public void AdicionarProduto(string descricao, decimal
valorUnitario, int quantidade)
    {
        ValorTotal += valorUnitario * quantidade;
    }

    public void FinalizarCompra(TipoPagamento tipoPagamento)
    {
        // Pagamento
        if (tipoPagamento == TipoPagamento.Debito)
            Console.WriteLine($"Pagamento no DÉBITO: R$
{ValorTotal:0.00}");
        else if (tipoPagamento == TipoPagamento.Credito)
            Console.WriteLine($"Pagamento no CRÉDITO: R$
{ValorTotal:0.00}");
        else if (tipoPagamento == TipoPagamento.Pix)
            Console.WriteLine($"Pagamento no PIX: R$
{ValorTotal:0.00}");
        else
            Console.WriteLine("Forma inválida!");

        // Comprovante
        Console.WriteLine($"Comprovante: compra no valor de R$
{ValorTotal:0.00}");
    }
}
```

Sua tarefa

Parte A — Aplicar SRP

Separe as responsabilidades do carrinho:

1. Carrinho deve cuidar apenas do **total da compra** e itens (se quiser)
2. Crie uma classe responsável por pagamento
3. Crie uma classe responsável por comprovante

O carrinho não deve imprimir nem decidir pagamento.

Parte B — Aplicar OCP

Remova `if/else` de forma de pagamento.

Crie uma abstração (interface ou classe abstrata) para pagamentos, por exemplo:

- `IPagamento` com `Pagar(decimal valor)` e `Nome`

Implemente pelo menos:

- `PagamentoDebito`
- `PagamentoCredito`
- `PagamentoPix`

Requisito OCP: adicionar uma nova forma de pagamento (ex: "Boleto") deve ser possível **sem alterar** classes já prontas (só criando nova classe).

Regras obrigatórias

- Não pode ter `if/else` ou `switch` dentro de pagamento para escolher forma
- `Program.cs` deve escolher qual pagamento usar (injeção manual)
- Deve rodar em Console App

Cenário de execução (obrigatório)

No `Program.cs`:

- Adicionar:
  - Mouse: 79,90 (2)
  - Teclado: 159,90 (1)
- Total esperado: `319,70`
- Executar com Pix e imprimir comprovante

Critérios de aceite

- O carrinho não sabe como paga nem como imprime
- Trocar Pix por Crédito exige mudar apenas o `Program.cs`
- Adicionar "Boleto" exige criar uma nova classe e não mexer em código pronto

Dica didática (pra você conduzir em sala)

Pergunte:

"Se amanhã surgir ApplePay, qual classe deve mudar?"

Exercício 2

Enunciado completo

Você precisa implementar um mini sistema de cadastro com:

- Produtos (Id Guid)
- Notas fiscais (Id int)

Você também precisa de um serviço de aplicação:

- `CadastroService` que cadastra produtos e notas

Requisitos

1) Entidades

Crie:

Produto

- `Guid Id`
- `string Descricao`
- `decimal ValorUnitario`

NotaFiscal

- `int Numero`
- `string Cliente`
- `decimal ValorTotal`

2) ISP — Interfaces segregadas

Você **não pode** criar uma interface gigante para repositório.

Crie as interfaces:

Interface base comum (sem Id específico)

- `IRepository<T>`
  - `void Inserir(T entidade)`
  - `IEnumerable<T> Listar()`

Repositório com Id Guid

- `IRepositoryGuid<T>`
  - `T BuscarPorId(Guid id)`
  - `void Atualizar(Guid id, T entidade)`

Repositório com Id int

- `IRepositoryInt<T>`
  - `T BuscarPorId(int id)`
  - `void Atualizar(int id, T entidade)`

Produto repositório implementa Guid
NotaFiscal repositório implementa int

3) DIP — Serviço de aplicação não depende de implementação concreta

Crie `CadastroService` que recebe dependências por construtor:

- `IRepository<Produto>` e `IRepositoryGuid<Produto>`
- `IRepository<NotaFiscal>` e `IRepositoryInt<NotaFiscal>`
- `ILogger`

O service não pode instanciar `new ProdutoRepositorio()` nem `new NotaFiscalRepositorio()`

4) Implementações (simples, em memória)

Implemente:

- `ProdutoRepositorioEmMemoria`
- `NotaFiscalRepositorioEmMemoria`

Eles devem armazenar dados em listas internas.

---

5) LSP — Substituição sem quebrar

Crie uma segunda implementação para Produto:

- `ProdutoRepositorioComValidacao`

Regra:

- Não pode quebrar o contrato do repositório.
- Exemplo de quebra (NÃO FAZER): lançar exceção em situações que antes não lançava sem justificativa.
- A validação deve ser coerente: ex.: impedir inserir produto com descrição vazia.

Se trocar `ProdutoRepositorioEmMemoria` por `ProdutoRepositorioComValidacao`, o sistema deve continuar funcionando, apenas recusando entradas inválidas de forma controlada.

6) SRP — Separar log e validação

Crie:

- `ILogger` + `ConsoleLogger`
- `IValidador<T>` + `ValidadorProduto` (opcional mas recomendado)

O `CadastroService`:

- chama o validador
- chama o repositório
- registra log

Cenário obrigatório (Program.cs)

1. Montar as dependências (injeção manual ou DI container, você escolhe)
2. Cadastrar:

- 2 produtos válidos
- 1 produto inválido (ex.: descrição vazia) e tratar falha sem "explodir" o app
- 2 notas fiscais

3. Listar produtos e notas ao final

Regras obrigatórias

- Sem `NotImplementedException` em método "obrigado pela interface"
- Interfaces pequenas (ISP real)
- `CadastroService` só depende de interfaces (DIP)
- Repositórios trocáveis sem quebrar fluxo (LSP)
- Sem misturar log/persistência/validação em uma classe só (SRP)

Critérios de aceite (o que você pode cobrar)

- Trocar repositório de produto por outro não exige mudar `CadastroService`
- Não há interface “gorda” com métodos de Guid e int misturados
- O sistema trata inválidos de forma previsível (sem comportamento surpresa)